basic tensor operations Assignment 2

Gabriel Severiche · 4 de octubre de 2026

Contenido

Khipus.ai

- Basic Tensor Operations
  - Assignment 2
  - Name: Gabriel Severiche
- Instructions
- Learning Objectives
- Basic Tensor Operations
- Converting Between NumPy and PyTorch Tensors

Khipus.ai

Basic Tensor Operations

Assignment 2

Name: Gabriel Severiche

© Copyright Notice 2025, Khipus.ai - All Rights Reserved.

Instructions

- Complete the following exercises using PyTorch.
- Read the descriptions provided before each code cell.
- Write your own code where prompted.
- Comment your code to explain its functionality.
- Ensure your outputs match the expected results where applicable.
- Submit the completed notebook.

Learning Objectives

By the end of this assignment, you will be able to:

- Perform basic mathematical operations on tensors.
- Convert between NumPy arrays and PyTorch tensors.

Basic Tensor Operations

Perform mathematical operations on tensors, such as addition, subtraction, multiplication, and division.

Expected output:

Sum: tensor([5, 7, 9])

Product: tensor([4, 10, 18])

Difference: tensor([-3, -3, -3])

Division: tensor([0.2500, 0.4000, 0.5000])

In [3]:

```
import torch
# Creating two tensors
# Create a tensor with values [1, 2, 3]
```

```
tensor_x = torch.tensor([1, 2, 3])
# Create another tensor with values [4, 5, 6]
tensor_y = torch.tensor([4, 5, 6])

# Element-wise addition
# Add tensor_x and tensor_y element-wise
tensor_sum = tensor_x + tensor_y
# Output the result of addition
print("Element-wise Addition:")
print(tensor_sum)

# Element-wise multiplication
# Multiply tensor_x and tensor_y element-wise
tensor_product = tensor_x * tensor_y
print("Element-wise Multiplication:")
print(tensor_product)

# Element-wise subtraction
# Subtract tensor_y from tensor_x element-wise
tensor_diff = tensor_x - tensor_y
print("Element-wise Subtraction:")
print(tensor_diff)

# Element-wise division
# Divide tensor_x by tensor_y element-wise
tensor_div = tensor_x / tensor_y
print("Element-wise Division:")
print(tensor_div)

# Your Task: Create two new tensors and perform matrix multiplication using torch.matmul.
```

Out [3]:

```
Element-wise Addition:
tensor([5, 7, 9])
Element-wise Multiplication:
tensor([ 4, 10, 18])
Element-wise Subtraction:
tensor([-3, -3, -3])
Element-wise Division:
tensor([0.2500, 0.4000, 0.5000])
```

Converting Between NumPy and PyTorch Tensors

PyTorch allows seamless conversion between NumPy arrays and tensors using `torch.from_numpy()` and `.numpy()`.

Expected Ouput:

NumPy Array: [100 150 200 250 300]

Converted Tensor: tensor([100, 150, 200, 250, 300], dtype=torch.int32)

Converted Back to NumPy: [100 150 200 250 300]

In [4]:

```
# Your Task: Create a NumPy, convert it to a PyTorch tensor, and then convert it back.
import numpy as np
# Creating a NumPy array
np_array = np.array([1, 2, 3, 4, 5])
# Output the NumPy array
print("NumPy Array:")
print(np_array)

# Converting NumPy array to PyTorch tensor
# Convert the NumPy array to a PyTorch tensor using torch.from_numpy()
pt_tensor = torch.from_numpy(np_array)
# Output the PyTorch tensor
print("PyTorch Tensor:")
print(pt_tensor)

# Converting back to NumPy
# Convert the PyTorch tensor back to a NumPy array using .numpy()
```

```
# Output the converted NumPy array
np_array_back = pt_tensor.numpy()
print("NumPy Array (Converted Back):")
print(np_array_back)
```

Out [4]:

```
NumPy Array:
[1 2 3 4 5]
PyTorch Tensor:
tensor([1, 2, 3, 4, 5])
NumPy Array (Converted Back):
[1 2 3 4 5]
```